

La récursivité

1. L'idée : les poupées russes

Imaginons une poupée russe. Pour atteindre ce qui se trouve dans la toute dernière poupée, nous devons :

1. Ouvrir la grande poupée.
2. À l'intérieur, nous trouvons une autre poupée. Elle est plus petite, **mais elle a la même forme**.
3. Nous répétons la même action jusqu'à trouver une poupée qui ne s'ouvre plus.

En programmation : une fonction récursive est une fonction qui **s'appelle elle-même** pour résoudre une version **plus petite** du même problème.

Pourquoi l'utiliser ?

La récursivité est utile quand un problème se décompose en **sous-problèmes de même nature**. L'exemple classique est l'exploration des dossiers d'un ordinateur : un dossier peut contenir des dossiers, qui peuvent eux-mêmes contenir des dossiers, etc. Une fonction récursive « explorer un dossier » s'écrit en quelques lignes, alors qu'une version avec des boucles est beaucoup plus complexe.

2. Les trois ingrédients d'une fonction récursive

Pour qu'une fonction récursive fonctionne et s'arrête, elle doit **obligatoirement** contenir :

1. **Le cas de base (condition d'arrêt)** : la situation la plus simple, pour laquelle on connaît la réponse **directement**, sans appel récursif. *Poupées : la poupée qui ne s'ouvre plus. Escalier : 0 marche.*
2. **L'appel récursif** : la fonction s'appelle elle-même avec un problème plus petit.
3. **La progression** : chaque appel doit **rapprocher du cas de base**. Si l'argument ne change pas, ou s'il s'éloigne du cas de base, la fonction ne s'arrêtera jamais.

Voici la forme générale :

```
TypeRetour foo(parametres)
{
    if (/* cas de base */)
    {
        return /* réponse directe */;
    }

    // appel récursif sur un problème plus petit, puis combinaison
    return /* ... */ foo(/* paramètres plus proches du cas de base */);
}
```

3. Premier exemple : le compte à rebours

```
#include <iostream>

void compteAREbours(int n)
{
    if (n == 0)                // cas de base
    {
        std::cout << "Décollage!" << std::endl;
    }
    else
    {
        std::cout << n << std::endl;    // action AVANT l'appel
        compteAREbours(n - 1);          // appel récursif, n se rapproche de 0
    }
}
```

compteAREbours(3) affiche :

```
3
2
1
Décollage!
```

Expérience : que se passe-t-il si on déplace l'affichage **après** l'appel récursif ?

4. La pile d'appels

Chaque fois qu'une fonction est appelée, l'ordinateur lui réserve un espace en mémoire qui contient **ses propres** paramètres et variables locales. Ces espaces sont empilés, comme des assiettes, dans une zone appelée **la pile d'appels** (*call stack*).

Deux règles gouvernent la pile :

- la fonction qui fait un appel **attend** que la fonction appelée ait terminé avant de continuer;
- seule la fonction du **dessus** de la pile s'exécute; quand elle se termine, elle est retirée et celle d'en dessous reprend là où elle s'était arrêtée.

Une exécution récursive se déroule donc en deux phases :

- **la descente** : les appels s'empilent jusqu'au cas de base;
- **la remontée** : les appels se terminent un à un, dans l'ordre inverse.

Ce qui est écrit **avant** l'appel récursif s'exécute pendant la descente; ce qui est écrit **après** s'exécute pendant la remontée. C'est pourquoi `compteCroissant` affiche les nombres à l'envers.

Exemple tracé : la somme des entiers

La fonction suivante retourne la somme des entiers de 1 à `nombre`. Par exemple, `sommeRecursive(5)` retourne $5 + 4 + 3 + 2 + 1 = 15$, et `sommeRecursive(-5)` retourne $-5 + -4 + -3 + -2 + -1 = -15$.

```
int sommeRecursive(int nombre)
{
    if (nombre == 0) // cas de base
    {
        return 0;
    }
    else if (nombre > 0)
    {
        return nombre + sommeRecursive(nombre - 1); // on se rapproche de 0 par le
haut
    }
    else // nombre < 0
    {
        return nombre + sommeRecursive(nombre + 1); // on se rapproche de 0 par le
bas
    }
}
```

Trace de `sommeRecursive(4)` :

DESCENTE (les appels s'empilent)	REMONTÉE (les résultats reviennent)
<code>sommeRecursive(4)</code> attend...	... reçoit 6, retourne $4 + 6 = 10$
<code>sommeRecursive(3)</code> attend...	... reçoit 3, retourne $3 + 3 = 6$
<code>sommeRecursive(2)</code> attend...	... reçoit 1, retourne $2 + 1 = 3$
<code>sommeRecursive(1)</code> attend...	... reçoit 0, retourne $1 + 0 = 1$
<code>sommeRecursive(0)</code> → cas de base, retourne 0	

On peut aussi l'écrire comme une suite de substitutions :

```
sommeRecursive(4)
= 4 + sommeRecursive(3)
= 4 + (3 + sommeRecursive(2))
= 4 + (3 + (2 + sommeRecursive(1)))
= 4 + (3 + (2 + (1 + sommeRecursive(0))))
= 4 + (3 + (2 + (1 + 0)))      ← cas de base atteint
= 10
```

Le débordement de pile (*stack overflow*) : la pile d'appels a une taille limitée (souvent quelques mégaoctets). Si les appels s'empilent sans jamais atteindre le cas de base, la pile se remplit et le programme plante. Cela peut aussi arriver avec une fonction **correcte** si elle fait trop d'appels imbriqués : sur une machine typique, `sommeRecursive(1000000)` suffit à faire planter le programme.

5. La méthode : concevoir une fonction récursive en quatre questions

Écrire une fonction récursive ne consiste pas à imaginer tous les appels dans sa tête. On répond plutôt à quatre questions :

1. **Quel est le cas le plus simple ?** C'est le cas de base : un problème si petit que la réponse est évidente.
2. **Comment rendre le problème plus petit ?** Par exemple : n devient $n - 1$, n devient $n / 10$, une chaîne perd son premier caractère, un tableau perd son dernier élément.
3. **Est-ce que toutes les entrées possibles mènent au cas de base ?** Vérifiez les nombres négatifs, la chaîne vide, les nombres impairs, etc.

Application : la puissance x^n

1. **Cas le plus simple :** $x^0 = 1$.
2. **Plus petit :** passer de n à $n - 1$.
3. **Construire la réponse :** si je connais x^{n-1} , alors $x^n = x \times x^{n-1}$.
4. **Toutes les entrées ?** Si n est négatif, $n - 1$ s'éloigne de 0 : la fonction ne s'arrêterait jamais ! On traite ce cas à part avec $x^{-n} = \frac{1}{x^n}$, ce qui nous ramène à un exposant positif, déjà résolu.

```
double puissanceRecursive(double nombre, int puissance)
{
    if (puissance == 0) // cas de base
    {
        return 1;
    }
    else if (puissance > 0)
    {
        return nombre * puissanceRecursive(nombre, puissance - 1);
    }
    else // puissance < 0 : on se ramène au cas positif
    {
        return 1.0 / puissanceRecursive(nombre, -puissance);
    }
}
```

Exemples : `puissanceRecursive(2, 10)` retourne 1024 et `puissanceRecursive(2, -3)` retourne 0.125.

Précondition : si `puissance` est négative, `nombre` ne doit pas valoir 0 (division par zéro).